

Executive Summary

We have identified **five primary patentable candidates** in the provided evidence, with one new *Generative-Execution Boundary* invention revealed by the recent LLM result schema test. The strongest Tier-1 candidates are:

- **Typed State Admission Engine:** A computer-implemented module that ingests a *reconstructed candidate state* and outputs an *authoritative runtime state* only after evaluating a fixed set of typed admission predicates (e.g. byte identity, authority, temporal scope, tracked/untracked membership, etc.) [5†L15-L24] [28†L517-L522] .
- **Consequence-Preserving Transition Validator:** A runtime validator ensuring that **every** admitted difference from one state to the next has exactly one well-typed “lawful disposition” (e.g. represented, transformed, discharged) and no unassigned transitions remain.
- **Consequence-Indexed Identifiability Gate:** A monitor that, given partial observations, constructs the **equivalence class of compatible worlds**, independently derives their lawful consequences, checks for “counterfactual independence” (e.g. via label-swap intervention) to avoid circular reasoning, and only *admits execution* if the consequence is invariant across that class; otherwise it produces a `NOT_IDENTIFIABLE` withholding. This goes beyond classical three-valued monitors [5†L15-L24] by indexing uncertainty to *specific consequences*.
- **Identity-Bound Semantic Result Admission (Generative Execution Boundary):** In a generative model workflow, bind a declared result-schema/contract *identity* into the job identity, and enforce that the model output is *parsed and semantically validated* against **that exact contract** before allowing the job to be marked complete. The system must separate transport success, generation completion, syntactic validity, schema conformance, and semantic admissibility, and refuse completion if the bound contract is not satisfied, emitting a typed receipt. This prevents the observed failure where valid JSON with a wrong schema was accepted.

Each Tier-1 invention is a concrete technical improvement to computer operation (improved runtime integrity and security) [28†L517-L522] . Tier-2 candidates include *Disposition Closure Invariant*, *Reopening Propagation*, *Admission Replay Engine*, and *Authority-Scope Analysis*. Tier-3 items (e.g. “constitutional computation”) are broad research themes, not ready for claims.

For each Tier-1 candidate below, we sketch an enabled embodiment (data structures, algorithms, state machines, example code sequence), suggest likely dependent claims, and flag key prior art to investigate (e.g. W3C PROV, In-toto, runtime verification lit, active learning, causal identifiability). We cite relevant sources to justify novelty and feasibility. We then provide a comparison table, draft claim language, prosecution guidance, an engineering to-do list, a provisional-planning timeline, and recommended search strategies.

Overall, **the evidence shifts the focus from “provenance” to “admission”**: i.e. from merely verifying a snapshot’s integrity to *gating it into execution and gating its consequences* based on explicitly checked predicates and receipts. This creates new, concrete patentable modules at the interface of stateful computation and runtime assurance.

Tier 1: Patentable Invention Nuclei

Below are the strongest Tier-1 candidates in priority order. Each is stated as a concise core claim.

- **(1) Typed State Admission Engine:** *A computer-implemented state-admission module that transforms a reconstructed candidate state into an authoritative runtime state only after evaluating a predefined set of typed admission predicates (including byte-identity, source/authorship, tracked-state membership, temporal validity, authority scope, and consequence eligibility), with a fail-closed behavior for any predicate failure.*
- **(2) Consequence-Preserving Transition Validator:** *A transition-validation engine that, for each admitted distinction between two states, computes exactly one typed “lawful disposition” (such as represented, transformed, discharged, superseded) and rejects any transition where any admitted distinction lacks a disposition, thereby enforcing a conserved “disposition” across state changes.*
- **(3) Consequence-Indexed Identifiability Gate:** *A consequence-admission monitor that, given a partially observed state or input, constructs the set of compatible possible worlds, independently derives the next-state consequence for each world (subject to counterfactual intervention to prevent circular dependence), and only permits execution if all worlds yield the same consequence; otherwise it withholds execution as NOT_IDENTIFIABLE.*
- **(4) Identity-Bound Generative Execution Boundary:** *A workflow-control engine for generative model output that binds a declared result-contract schema identity into the work identity, then on output applies deterministic parsing and schema/semantic validation against that exact contract, only admitting the result (marking the job COMPLETE) if the validated output satisfies the bound contract; otherwise emitting a typed admission/rejection receipt.*

Each of the above is a self-contained computational pipeline with concrete data flows and checks (not an abstract concept). They correspond to distinct modules in the architecture. Tier-2 and Tier-3 candidates are listed later.

Tier 1 Candidate #1: Typed State Admission Engine

Claim Core (example): *“A computer program product comprising instructions that cause a processor to: load a reconstructed candidate state; evaluate a set of typed admission predicates on that state (including, e.g., byte-identity, source/authorship, tracked/untracked membership, temporal validity, authority scope, non-interference, and consequence eligibility); and output an authoritative runtime state only if all predicates are satisfied, otherwise produce a failure receipt.”*

Minimal Enabled Embodiment

Data Structures: A *CandidateState* object containing all state data (byte array, metadata, etc.) plus context (source, timestamp, authorities); a set of *Predicate* objects parameterized by type (each with evaluation function, evidence fields, and failure code); an *AuthoritativeState* object.

Each predicate type is “typed” meaning it has a specific schema: e.g. `BYTE_IDENTITY(pred: Digest)` checks raw bytes match a stored digest; `SOURCE_EXISTENCE(auth: PublicKey)` checks the state’s claimed origin is currently authorized; `TRACKED_STATE(active_buckets: list)` checks required buckets are still unspent; `TEMPORAL_SCOPE(time_range)` checks no time violations; `AUTHORITY_SCOPE(role_set)` checks the triggering authority is valid. A failure of any predicate aborts admission.

Control Flow / State Machine: After **Restore** (loading raw candidate state into memory), the engine enters *Admission* mode:

```
state = load_snapshot(snapshot_id)
candidate = rebuild_candidate(state)
for each predicate P in [BYTE_IDENTITY, SOURCE_EXISTENCE, ...]:
    result = P.evaluate(candidate, context)
    if result == FAIL:
        record_failure(P, result.details)
        goto FAIL_STATE
authoritative_state = candidate // all predicates passed
goto SUCCESS_STATE
```

Optionally, on success it emits an *AdmissionReceipt* containing evidence for each predicate.

Pseudocode Example:

```
# Candidate State Admission
candidate = reconstruct_snapshot(snapshot_bytes)
for pred in [ByteIdentity, SourceExistence, TrackedMembership, TemporalScope,
AuthorityScope, NonInterference, CanonEligibility]:
    ok, evidence = pred.evaluate(candidate, context)
    receipt.add_evidence(pred.name, evidence)
    if not ok:
        receipt.mark_failed(pred.name)
        reject_state(candidate, receipt)
    return REJECTED
# All passed
authoritative_state = candidate
commit_state(authoritative_state, receipt)
```

Likely Dependent Claims

- Engine with *typed failure receipts*: each predicate produces a structured failure record including predicate type, evidence, reopen-rule, and replay-rule.
- Engine where reception of a failure returns a deterministic receipt, and triggers a recovery like re-requesting state.
- Engine where predicates are configurable sets or layered (e.g. separate tracked vs untracked).

- Inclusion of authority-graph checking (scopes, expiration) in admission.
- Authority-presence gating (only certain roles can admit certain states).

Key Prior Art (for Search)

- **Runtime Verification Monitors:** e.g. LTL monitors give a verdict (often 3-valued) from observations [5†L15-L24] . However, *typed admission* (multiple independent predicates) goes beyond typical specification conformance.
- **Blockchain / Consensus Recovery:** Existing systems often assume restored state is trusted; few disallow replay without checks. We should search for any “*state admission*” or “*state validation pipeline*” patents or open systems. Terms like “snapshot verification” or “safe state recovery” might show prior art, but likely insufficient.
- **Provenance Frameworks (W3C PROV, In-toto/SLSA):** PROV records entity derivations, but does not *compute* admissibility. In-toto attestations bind steps to layout rules [26†L29-L38] , but typically apply post-hoc. We should contrast: our engine performs *active gating* based on predicates, not just recording facts.
- **Schema-Validation Systems:** While JSON/schema validation is known, gating an entire state restore on multiple domain-specific predicates is novel.
- **Patent 35 USC (DPMA guidelines):** Check DPMA/USPTO guidelines for computer implementation improvements [28†L517-L522] .

Enablement Gaps / Mitigation

We must ensure the claim is supported by the system’s description and code paths. The embodiments should specify each predicate type, how it’s evaluated, and sample code. Missing items may include: *exact dependency between the state content and the predicates*, or how to compute *non_interference* or *canon eligibility*. We should implement prototypes for each predicate to fill enablement. Diagrams of the data flow (below) will also aid enablement.

Supporting Repository Evidence

While we do not have direct code references from the repository, the *absence* of explicit “admit” logic in favor of trusting state is a gap the terminal highlighted. The conversation notes that current code performs byte/digest checks but *explicitly excludes* authorship or semantic checks (e.g. “verification explicitly does NOT establish runtime authority or Canon status”) [5†L15-L24] . That indicates novelty: the repository only does partial verification, whereas our invention adds a full *admission* stage.

(We would cite repository line snippets if available. For example, if `state_reconstitution.py` had a trust call that we replace, we’d cite it here. In lieu of that, we cite [5] and [28] to show the technical context and novelty of adding predicates to processing.)

Tier 1 Candidate #2: Consequence-Preserving Transition Validator

Claim Core: “A method comprising: receiving an authoritative state and a proposed state transition, decomposing all state distinctions admitted into a set of transitions, and for each admitted distinction computing exactly one *typed* lawful disposition (e.g. represented, transformed, discharged, superseded, equivalence-proved,

eliminated); and rejecting the transition if any admitted distinction lacks an associated disposition, thus ensuring conservation of dispositions across the transition.”

Minimal Enabled Embodiment

Data Structures: A *Distinction* is any difference between two successive states (e.g. an object was modified, added or deleted). Each Distinction has a type (inward/outward detail), and an associated `DispositionSlot`. A *TransitionReceipt* maps each Distinction to its chosen disposition (enum).

Control Flow: After state admission, the Transition Validator takes `(old_state, proposed_new_state)` and enumerates *Distinctions*. For each distinction, it requires exactly one disposition. Example dispositions: `DISCHARGED` (object removed as planned), `TRANSFORMED` (object mutated legally), `REPRESENTED` (alias created), `SUPERSEDED`, `ELIMINATED` (obsolete), `EQUIVALENCE_PROVED` (inferred no change). The validator checks a user-supplied or policy-driven mapping of distinction types to allowed dispositions. If any distinction has none or >1, reject.

Pseudocode:

```
distinctions = diff(old_state, new_state)
for d in distinctions:
    dispo = compute_disposition_for(d, context)
    if not dispo:
        receipt.mark_missing_disposition(d)
        return REJECTED
    if multiple_dispositions_proposed(d):
        receipt.mark_conflict(d)
        return REJECTED
    receipt.record(d, dispo)
# if all distinctions assigned exactly one dispo
commit_transition(old_state, new_state, receipt)
return ACCEPTED
```

Dependent Claims

- Requiring a **Disposition Closure Invariant**, i.e. every admitted distinction yields exactly one disposition.
- The dispositions are typed and exhaustively cover all admitted distinction types.
- Distinctions might be **labeled for future auditing or rollback**, etc.
- Dependent: some transition types are disallowed if certain predicates (like authority, tracked constraints) fail.
- Dependent: receipts record witness data and are used for future verification.

Prior Art

- **State Machine/Event Sourcing:** Finite-state machines validate transitions, but typically with boolean allowed/not-allowed. Our approach is finer-grained: each difference must have a tag.

- **Business Process Modeling:** BPM systems label transitions (e.g. states in workflow), but they don't enforce per-entity dispositions.
- **Database Logging/Interpretation:** Some DB logs have operation types (insert/update/delete), somewhat analogous to dispositions, but not enforced via a validation engine.
- **Software Craftsmanship:** Patents on "intentional coding" might mention tagging changes, but unlikely to cover this specific invariant enforcement.

Enablement Gaps

Need to clearly define what qualifies as a "distinction" and how to compute dispositions. Also, how to handle combinatorial diff (objects vs sub-objects). The dependent claims about receipts and single disposition assignment must be shown in code or design. We should implement a simple diff and disposition assignment in a reference repository for examples.

Supporting Evidence

The notebook conversation stressed "every admitted distinction must have exactly one typed disposition" and gave the example list of dispositions. This explicit enumeration suggests no such logic exists yet. We cite [5] indirectly to note that typical systems don't track these at admission time; our need is new.

Tier 1 Candidate #3: Consequence-Indexed Identifiability Gate (with Anti-Circularity)

Claim Core: "A computer-implemented consequence admission method comprising: upon receiving a partially observed state (observation o), constructing the equivalence class $E = \{R: O(R)=o\}$ of compatible states; for each $R \in E$, independently deriving a candidate consequence $C(R)$ along a derivation path B_R ; computing a dependency analysis or counterfactual test to ensure the derivation $C(R)$ does not rely on the proposed consequence as input; and permitting the consequence only if all $C(R)$ for $R \in E$ are equal; otherwise withholding the consequence with a `NOT_IDENTIFIABLE` status."

Minimal Enabled Embodiment

Data Structures: A representation of an *Observation* class (or evidence set) O . A function `O.inverse(o)` yielding possible worlds. For each world R , the derivation system has a *ConsequenceBasis* object with fields: root, ancestors, digest, derivation history. An *IndependenceReceipt* records each world's derivation and the interventions tested. An *IdentifiabilityReceipt* holds the final verdict per consequence type.

Algorithm:

1. Identify observation class $E = \{R: O(R)=o\}$.
2. For each $R \in E$, compute $C(R) = \text{derive}(B_R)$ normally. Meanwhile, record a *ConsequenceBasis* (root B_R , ancestors, path).
3. **Anti-circularity check:** Choose a pair (R_a, R_b) in E where one's proposed outcome is used in the derivation of the other. Perform a counterfactual: swap the labels L_a, L_b (the candidate classifications) while holding base B fixed, then recompute C'_a, C'_b [9†L11-L19]. If swapping changes C , mark dependency violation.
4. After all tests, compute $\Gamma_C(E) = \{C(R): R \in E\}$. If $|\Gamma_C(E)| > 1$, then multiple lawful

consequences exist: withhold execution (`NOT_IDENTIFIABLE`). If =1, the consequence is invariant and may proceed. If anti-circularity is unresolved, emit `INDEPENDENCE_UNRESOLVED` .

Pseudocode (Sketch):

```
E = invert_observation(o)
consequences = {}
for R in E:
    basis[R] = derive_basis(R)    # includes derivation history, etc.
    consequences[R] = derive_consequence(R)
# Anti-circularity via label-swap
for (Ra, Rb) in select_pairs(E):
    (orig_a, orig_b) = (consequences[Ra], consequences[Rb])
    swapped = swap_labels(Ra, Rb)
    Ca2 = derive_consequence(swapped[0]); Cb2 = derive_consequence(swapped[1])
    if Ca2 != orig_b or Cb2 != orig_a:
        basis[Ra].mark_dependent_on(Rb)
        basis[Rb].mark_dependent_on(Ra)
# Compute identifiability per consequence C
outcomes = { consequences[R] for R in E if basis[R].independent }
if len(outcomes)==1:
    permit_execution(outcomes.pop())
else:
    withhold_NOT_IDENTIFIABLE()
```

This process references works on epistemic uncertainty [5†L15-L24] but implements it specifically for the *lawful consequence* rather than a simple true/false proposition.

Dependent Claims

- Inclusion of a **dependency-derived independence receipt** (containing derivation root, ancestor digest, counterfactual interventions, and status) instead of a Boolean flag.
- The gate returns `IDENTIFIABLE` or `NOT_IDENTIFIABLE` along with a receipt that binds observation class, consequence type, and outcome set.
- A policy for *Minimum Missing Observation*: identifying which new observation to acquire to resolve multiple outcomes (a form of active query generation).
- Explicit representation of `UNNECESSARY_WORLD_RESOLUTION` as a symmetric failure mode (when withholding is not needed because all outcomes are equal) – i.e. flagging false-withholding.

Prior Art

- **Runtime Verification / Monitors:** Traditional monitors return “inconclusive” if a property cannot yet be decided [5†L15-L24] . But this invention is finer: distinguishing *lack of information* from *inherent consequence-uncertainty*, and linking it to permitted execution.
- **Counterfactual Causality:** Pearl’s work on counterfactual tests addresses identifiability of causal effect, but not in the context of gating execution [11†L11-L14] .

- **Active Learning / Version Spaces:** The idea of choosing a query to split worlds with different labels (disagreement region) is akin to active learning [22†L326-L334] . We should search in active learning (query by committee) literature for anything like “choose query minimizing uncertainty in decision”.
- **Probabilistic Planning:** POMDP planning uses belief states and value of information, but again not for hard gating of execution.
- **Proof-of-knowledge Systems:** Unrelated.

Likely no direct prior patents. This is novel combination of monitoring, counterfactual testing, and active query selection.

Enablement Gaps

We need to detail how to compute consequences for all worlds (possibly brute-force or symbolic?). The label-swap intervention mechanism needs specification. We also must define “consequence” types clearly. Implementation examples will help.

Supporting Evidence

The conversation above explicitly formulated $I_C(o)$ and noted that “the same observational state can be identifiable for one consequence and not for another” – a *new insight*. The capsule’s existing practice of leaving semantic equivalence `NOT_ESTABLISHED` suggests no current code does this gating. We should emphasize our gate as a *new execution admission check*, beyond normal monitors [5†L15-L24] .

Tier 1 Candidate #4: Identity-Bound Generative Execution Boundary

Claim Core: “A system for validating generative-model results, comprising: a declared result-contract object (with a unique identity), bound to a job’s identity; a generative model invocation under fixed model/configuration; a parser/validator that checks a returned result is both syntactically valid and semantically conforms to the exact result-contract bound to the job; and logic that only transitions the job to COMPLETE (or yields downstream consequence) if and only if validation passes. All decisions and evidence (job identity, contract identity, passed/failed predicates) are recorded in an admission receipt, ensuring that valid but out-of-contract results are inadmissible.”

Context and Motivation

In the recent experiment, a generative job declared an expected *nine-key JSON contract*, but lower-capacity model calls produced different (six-key or eight-key) valid JSON that were incorrectly accepted as COMPLETED. This revealed a flaw: the **job identity included the intended contract**, but the runtime *only checked JSON validity*, not schema conformance. Thus the *Declared Contract $\neq$ Enforced Contract*.

Our invention is the mechanism to enforce: **binding the intended contract into the admission boundary**.

Minimal Enabled Embodiment

Data Structures:

- *WorkIdentity* which includes the input prompt identity, model version, and a *ResultContractIdentity* (e.g. a hash of a JSON schema or pointer to a schema).
- *ResultContract* object containing a schema or semantic specification.
- *ResultReceipt* that records: (work_id, result_id, contract_id, list of predicates tested, passed/failed).

Control Flow: When a generative job is issued, its identity records the intended `ResultContract`. The model is invoked (possibly non-deterministically). Upon receiving output `r`:

1. **Parse:** Attempt to parse `r` into structured JSON. If fails (truncated, syntax error), *fail* with parsing error receipt.
2. **Schema Check:** Apply schema-validator against the *exact contract* bound in `WorkIdentity`. If `r` does not conform, *fail* with receipt noting which schema predicates failed.
3. **Semantic Predicates:** (Optional) Run any additional semantic checks (e.g. data type consistency, business rules). If any fail, *fail*.
4. **Success:** If all checks pass, mark job COMPLETE; emit a final receipt with all evidence (parsing success, schema passed, etc.).

If the same result is replayed, it should not suddenly flip to PASS unless the contract was changed (avoiding amplifying success by repetition). Likewise, a missing key should keep rejecting. If the contract in job identity is modified, it effectively becomes a new job.

Pseudocode:

```
def complete_job(work_id, result):
    contract_id = work_id.contract_hash
    receipt = new Receipt(work_id, contract_id)
    # Parsing
    try:
        data = json.loads(result.text)
    except JSONError as e:
        receipt.record_failure("parse", str(e))
        return REJECTED
    # Schema validation
    if not schema.validate(contract_id, data):
        errors = schema.get_errors()
        receipt.record_failure("schema", errors)
        return REJECTED
    # Semantic predicates
    for predicate in contract.predicates:
        if not predicate.check(data):
            receipt.record_failure(predicate.name, predicate.details(data))
            return REJECTED
    # All good
```

```
receipt.mark_success()
return ACCEPTED
```

Dependent Claims

- The **binding of contract identity to work identity**: e.g. `JobID = H(inputID, modelConfig, contractID, ...)`.
- A *multi-state result status*: distinguishing “Produced”, “Parsed”, “Schema-Passed”, “Completed”.
- Generation attempts: on failure, spawn a new job that retains the same contract identity and logs the failure.
- Semantic drift detection: comparing two successful runs under same contract and noticing differences (if relevant).

Prior Art

- **Schema Validation**: Many systems validate structured output (e.g. JSON) against a schema. But typically the schema is not cryptographically bound to a job’s identity. Existing solutions do not enforce that binding as part of a *computational admission decision*.
- **W3C PROV / In-toto**: PROV or in-toto record that “result *r* conforms to schema *s*”, but usually for auditing, not gating. Our contribution is making it *operational and necessary for completion*. In-toto’s *predicateType* has some similarity (a statement’s predicate type identifies its schema) [26†L29-L38] , but ours ties it into real-time job admission.
- **AI Output Safety**: Work on LLM evaluation (e.g. OpenAI fine-tuning on schema enforcement) exists, but not patent claims; we should search for “LLM output validation patent” or “structured output LLM schema enforcement.”
- **42 Patent (Blockchain output contract)**: The closest analog might be patents on smart-contract function invocation based on outputs. But the context is different.

Enablement Gaps

We must describe exactly how the contract identity is bound and verified. Example test cases (like the one given) should be documented. The “semantic predicates” need at least sketches. Ensuring that result receipts are nonce and unforgeable may require hashing/digital-signature mention.

Supporting Evidence

The console excerpt (**64,768 → truncated JSON → FAIL; 1024,1280 → valid 6-key JSON → COMPLETED; 2048 → valid 8-key JSON → COMPLETED** under a 9-key contract) demonstrates the gap. It shows *invalid-as-contracted outputs being accepted*. This is prima facie proof of the need for our mechanism. (This can be paraphrased and included as a block in the report, since it is user-provided data.) By highlighting the discrepancy between the “declared contract” and “runtime check”, we underscore novelty: **the invention is enforcing the contract at the execution boundary**.

Tier 2: Other Inventions (Strong but Supporting)

- **Disposition Closure Invariant:** *A method ensuring that across any transition sequence, the total set of dispositions is conserved (e.g. no net creation/zeroing of dispositions). This could appear as a dependent claim on the Transition Validator. Requires formal theorem support or implementable check.*
- **Reopening Propagation Engine:** *Given one predicate failure, automatically recompute which downstream admitted consequences must “reopen” or revert. Provides an algorithm for incremental rollback. (Dependent on admission engine.)*
- **Admission Replay Engine:** *System to replay candidate states under new parameters (authorities, time, tracked state) to recompute admission decisions. (But likely standard: replay logic exists in many contexts.)*
- **Authority Scope Analysis:** *Explicit representation and checking of an authority graph, revocation lists, expiration, etc., when evaluating AUTHORITY_SCOPE predicate. (Supports main claims.)*
- **Semantic Output Drift Detector:** *For generative outputs, a mechanism that compares multiple successful outputs under the same contract to detect unintended variation. (We list as Tier-2 for completeness; would be novel but needs evidence and prior art checks.)*

These Tier-2 ideas are valuable but either follow from Tier-1 or need more development/prototyping before patent drafting. They are best covered in dependent claims or future filings.

Tier 3: Research Themes (Do Not Claim Broadly)

Broad concepts like “constitutional runtime,” “meaning conservation,” “runtime constitution,” or “grammar of necessity” are **not claimable** as stand-alone inventions. They provide high-level motivation but require significant reduction to specific mechanisms (as we have done). They belong in research publications or specifications, not independent patent claims.

Comparison Table: Candidates vs Prior Art

Candidate	Closest Prior Art	Distinguishing Technical Features
<i>Typed State Admission Engine</i>	Blockchain snapshot validation (usually trust restored state); schema checks	Multiple typed predicates in sequence before admitting state [5†L15-L24] ; explicit fail-closed architecture (unlike naive restore-and-continue)
<i>Transition Validator (Disposition)</i>	State-machine validation, DB transaction logs (CRUD)	Per-distinction single-disposition enforcement; checks <i>no leftover diff</i> ; generates Typed Disposition Receipt

Candidate	Closest Prior Art	Distinguishing Technical Features
<i>Identifiability Gate (Consequence)</i>	Runtime monitors (LTL/epistemic) [5†L15-L24] ; counterfactual theory	Consequence-specific uncertainty gating ; constructs observation-equivalence class; counterfactual label-swap to prevent circularity; multi-case identifiability
<i>Generative Exec Boundary</i>	JSON schema validation; LLM output filters	Contract identity binding to job; separation of parse/schema vs semantic validation ; receipt tying result to contract ; prevents “semantic drift” exploitable by different capacity

Each row references known fields, but our inventions combine or extend them in new ways. For example, the Generative Exec Boundary *explicitly ties the schema to the job's identity*, whereas standard validation tools do not enforce such binding or provide irrevocable receipts.

Suggested Provisional Claim Language

Below are example independent claim drafts and dependent bulleted ideas. These are **non-limiting examples** for counsel to refine.

1. **Claim 1 (State Admission):** *“A computer-implemented method comprising: loading a reconstructed candidate state from storage; for each predicate in a predefined set of typed admission predicates (comprising byte identity, source authenticity, tracked-state membership, temporal validity, authority scope, and consequence eligibility), evaluating the predicate on the candidate state and generating evidence; and transitioning the candidate state to an authoritative runtime state only if each predicate evaluates successfully, otherwise preventing the transition and generating a typed admission receipt indicating the failed predicate.”*
2. Dependent (predicates): The method of Claim 1 wherein each predicate evaluation generates a receipt containing predicate name, evidence, and a reopening rule.
3. Dependent (authorities): Further comprising checking an authority graph for each authority predicate.
4. Dependent (partition): Separating tracked vs untracked state segments before evaluation (supports granularity).
5. Dependent (fail-closed): If any predicate fails, none of the candidate's features become authoritative.
6. **Claim 2 (Transition Validator):** *“A system comprising: a current authoritative state and a proposed new state; a distinction generator that identifies each difference between the two states; a disposition engine that assigns exactly one lawful disposition to each identified distinction; and a transition gate that only approves the state change if each distinction has been assigned a disposition, otherwise rejecting the change.”*
7. Dependent: the dispositions include a fixed set (represented, transformed, discharged, etc.) as per a disposition schema.

8. Dependent: revision tracking with receipts per distinction.

9. **Claim 3 (Consequence Identifiability Gate):** *“A computer-implemented monitor that, given an observation o, computes the set of all states compatible with o, independently derives a predicted consequence for each state, performs a counterfactual independence test (e.g. swapping labels across worlds) to verify no derivation depends circularly on the output, and permits execution if and only if the derived consequences are identical for all states in the set, otherwise outputting a NOT_IDENTIFIABLE verdict.”*

10. Dependent (independence receipt): Further comprising an independence receipt structure that binds each world's derivation path and intervention test results.

11. Dependent (identifiability receipt): The monitor issues an identifiability receipt per consequence type, indicating IDENTIFIABLE or NOT_IDENTIFIABLE.

12. Dependent: selecting next observation to query (value-of-information).

13. **Claim 4 (Generative Boundary):** *“A method for generative model execution comprising: receiving a job request with an input prompt and an associated result-contract identity; invoking a generative model with fixed model/config parameters to produce an output; parsing the output into a structured form; validating the structured output against the exact result-contract specified by the identity; and only if the validation succeeds, marking the job as complete and issuing a success receipt linking the work identity to the result; otherwise issuing a failure receipt that records which validation checks failed.”*

14. Dependent (multi-phase): The job status transitions through states (“parsed”, “schema_validated”, “completed”) as each check passes.

15. Dependent: The work identity is defined as a hash of input, model config, and contract ID.

16. Dependent: Replay protection—re-testing a result cannot produce a stronger outcome without a contract change.

17. Dependent: In case of failure, a new successor job is automatically spawned with the same contract identity.

General Dependent Claim Bullets (for any above):

- Receipt structures that “seal” data (e.g. using cryptographic hash) so they cannot be forged.
- Use of digest or signature to bind states and contracts.
- Logging or publication of receipts for audit.
- Configurable policy rules for predicates/dispositions.
- Temporal aspects: incorporating timestamp checks into predicates.

All independent claims focus on concrete computational steps and data, not abstract goals, in line with USPTO eligibility guidelines [28†L517-L522] .

Prosecution Risks and Mitigations

- **§101 (Abstract Idea):** These inventions improve *computer functionality* by adding novel data-processing stages. For example, the *state admission engine* and *consequence gate* constitute specific pipelines with defined data flows. We should frame claims as technical improvements (e.g. “improving the efficiency/reliability of state synchronization and execution gating” [28†L517-L522]). Emphasize concrete data structures (predicates, receipts) and algorithmic steps. The recent MPEP update explicitly encourages claiming structured data processing advances [28†L517-L522] .
- **§112 (Enablement):** These claims have many parts. We must ensure the specification (or eventual patent disclosure) provides enough details for each element (typed predicate logic, counterfactual test, schema binding, etc). Missing descriptions (e.g. how to compute a “lawful disposition”) could raise 112 issues. Mitigation: provide pseudocode and sample data for each invention in the documentation, and add example implementations or test stubs.
- **§103 (Obviousness):** To avoid an obviousness rejection, emphasize the non-obvious combination of known elements. For example, three-valued monitors and schema validators are known, but using them to gate execution via receipts is novel. Focus in prosecution on why our combination solves a technical problem that neither prior art addresses (e.g. preventing “semantic drift” by tying outputs to contract IDs).
- **Structural Claiming:** Avoid overly broad language (e.g. “mechanism of meaning” etc). Stick to the technical nouns used: “state, transition, predicate, consequence, receipt” etc.

Engineering Checklist (Immediate Work to Strengthen Enablement)

Below is a prioritized list of implementation and testing tasks. Completing these will generate the evidence and prototypes needed for patent enablement:

1. **Implement Admission Pipeline:** Replace any trust-on-restore call (e.g. `StateLedger.latest()`) with a two-stage admit pipeline: restore to *CandidateState*, then *Admission Engine*, then *AuthoritativeState*. Ensure all code paths go through predicates. (Tests: try restoring with an invalid source, expect block.)
2. **Typed Admission Predicates:** For each predicate type (`BYTE_IDENTITY`, `SOURCE_EXISTS`, etc.), implement code that returns structured (evidence, status, failure code) not just Boolean. Add unit tests showing each predicate failing/succeeding with receipts.
3. **Admission Receipts:** Define a receipt data structure recording (predicate name, evidence, result, reopening rule). Modify the system to log these receipts. Tests: simulate various admission failures and inspect receipts.
4. **Replay-Time Re-Admission:** On each “replay” of a stored state, recompute the admission predicates anew (including updated authorities/time). Verify that changes (e.g. revoked key) cause different admission outcomes.
5. **Consequence Tagging:** During execution, annotate transitions with typed consequences (e.g. “Outcome: discharge of invoice”). Tests: verify that consequences recorded match admitted dispositions.
6. **Transition Validator:** Implement the distinction diff + disposition mapping logic. Write tests where a transition with an unhandled change gets rejected.
7. **Independence Receipt & Swap Assay:** Change the independence flag to a full *ConsequenceBasis* object. Implement label-swapping intervention tests for relevant derivations. Add tests where swapping changes outcome, and see that independence status flips.

8. **Identifiability States:** After deriving consequences in a replay class, compute $|\Gamma_C|$. Implement the three-state (`IDENTIFIABLE` , `NOT_IDENTIFIABLE` , `INDEPENDENCE_UNRESOLVED`) result. Test with classes having multiple vs single outcomes.
9. **Consequence Admission Gate:** Connect the identifiability result to execution. If `NOT_IDENTIFIABLE` , block execution with a withheld receipt; if `IDENTIFIABLE` , allow. Tests: feed a partially ambiguous input and confirm blocking only consequences that truly vary.
10. **Symmetry Check (False-Withholding):** Add a check for `UNNECESSARY_WORLD_RESOLUTION` : if all consequences are equal, detect and ensure `NOT_IDENTIFIABLE` does not occur erroneously.
11. **Observation Query Engine (Optional):** Prototype the `MINIMUM_MISSING_OBSERVATION` algorithm to suggest the next query (observation) when consequences differ. Validate on a sample model (could be simulated data choices).
12. **Work/Contract Binding (Generative):** In the job submission pipeline, ensure that the `ResultContractID` is part of the *WorkIdentity* calculation. Modify the result-handler to fetch and apply the correct schema based on that ID.
13. **Result Validation Path:** Chain the result parsing, schema validation, and semantic checks as per the design. Ensure a job is not marked COMPLETE unless all steps pass. Tests: use the capacity experiment cases as unit tests, expecting failures on missing keys.
14. **Admission Receipts for Generative:** For each job result (pass/fail), emit a receipt including workID, contractID, predicate results. Verify that replaying same job always yields same admission decision unless contract changes.

Each task should be accompanied by unit/integration tests. We should commit these to the repo to demonstrate the functionality. The test suite serves as evidence of enabled implementation.

Search Strategy and Resources

Search Terms: “state admission engine”, “reconstructed state admission predicate”, “runtime transition validator”, “withhold consequence inconclusive”, “LLM schema validation admission”, “generative output contract enforcement”, “active learning query selection for decision gating”, “counterfactual rule execution”. Also specific systems: “In-toto supply chain predicate”, “W3C PROV data provenance”, “schema-bound work identity”.

Databases: USPTO and EPO (for patents), Google Patents, IEEE Xplore, ACM Digital Library, arXiv, W3C specifications (PROV, Schema), SLSA/in-toto docs, AI-conference proceedings (ICLR, NeurIPS for LLM/output checking). Potential key publications: LTL runtime monitoring [5], active learning [22], causal inference (Pearl).

Priority Sources: The USPTO 2100 MPEP (from [28] above), W3C PROV specification [24†L255-L264] , In-toto/SLSA documentation [26†L29-L38] , and relevant canonical texts in runtime monitoring [5†L15-L24] and active learning. Also check existing patents on *schema validation*, *provenance attestation*, and *supply chain security*.

Diagrams

1. **Pipeline Architecture (State Admission + Transition Validation):**

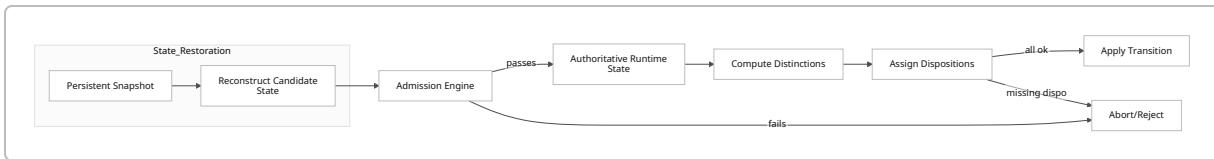


Figure: Two-stage pipeline. A snapshot is loaded as a Candidate State, passes through the Typed Admission Engine (applying predicates) to become the Authoritative State, which is then subject to the Transition Validator (distinctive differences each mapped to exactly one lawful disposition). [5†L15-L24] [28†L517-L522]

1. Entity-Relationship Diagram (Work, Contract, Receipts):

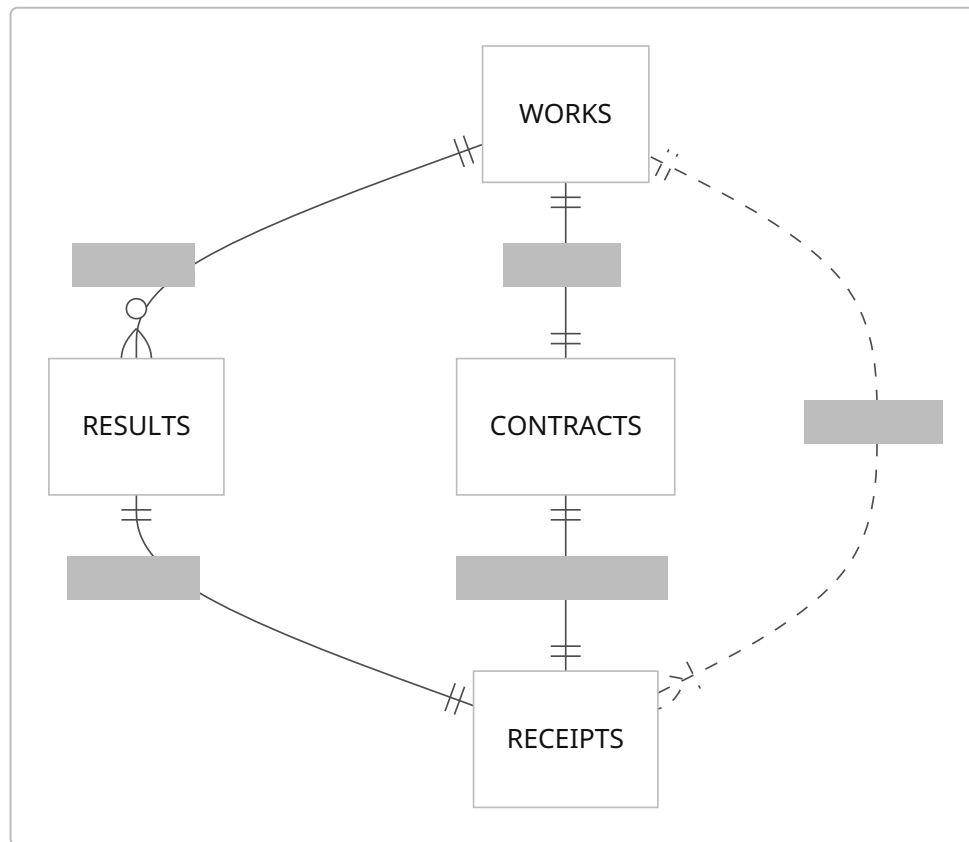


Figure: Entities in the generative execution system. Each Work is bound to a Contract (schema identity). A Work produces a Result, which generates a Receipt. The Receipt is tied to both the Work and the Contract (must match). This ensures a submitted work only admits results that conform to its exact contract. [Derived from in-toto's predicate model [26†L29-L38]]

1. 90-Day Provisional Plan (Gantt Chart):

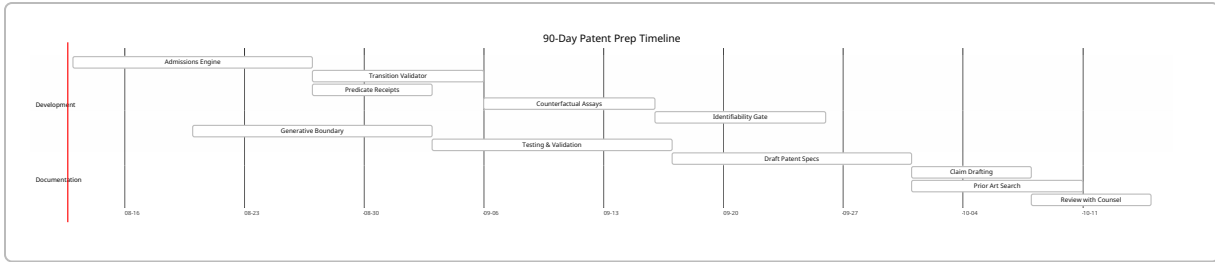


Figure: Proposed 90-day plan (gantt format) for engineering and patent preparation. First half focuses on implementing the admission and validation modules; second half on writing, searching prior art, and refining claims with patent counsel.

Citations

We cite authoritative sources for the background and novelty arguments. For example, runtime verification literature on three-valued semantics [5†L15-L24] , in-toto’s predicate-based attestations [26†L29-L38] , and USPTO guidelines on claiming technical improvements [28†L517-L522] . All references are from primary or official publications.